01 / OVERVIEW

Technical Documentation

2026 FinTechathon International Track
Topic E - SME Finance Copilot

Purpose. Run the shop and use its records to make financial decisions. BizWizard combines business management with financing comparisons, cash-flow alerts and financial analysis. Code calculates the numbers; optional AI helps choose or explain allowed options.

The business records behind the copilot

Inventory, sales, purchases, customers, suppliers, debts, expenses and employee workflows share one platform. The finance tools use records saved through these workflows rather than asking owners to build a separate financial dataset. Demonstrations and submitted test evidence use synthetic data, not verified customer records.

Implemented capabilities

Topic E capability	Implemented workflow
Financing-option matching	Build a shop profile; apply eligibility rules; rank feasible products; price and negotiate simulated offers; create an owner-approved application and repayment schedule.
Cash-flow management	Maintain cash events and commitments; project a 90-day balance; raise rule-based alerts; propose guarded treasury actions and link recorded collection evidence.
Business financial analysis	Compute product economics and working-capital metrics; verify candidate explanations; compare scenarios; measure 30-day observations and matched sandbox outcomes.

Assessment criteria

Task completion (40%): the three workflows and their execution evidence. **Security & compliance (30%):** authorization, financial guards and risk assessment. **Innovation & interaction (30%):** structured AI choices, policy-based negotiation, editable mandates and reproducible paired evaluations.

Scope

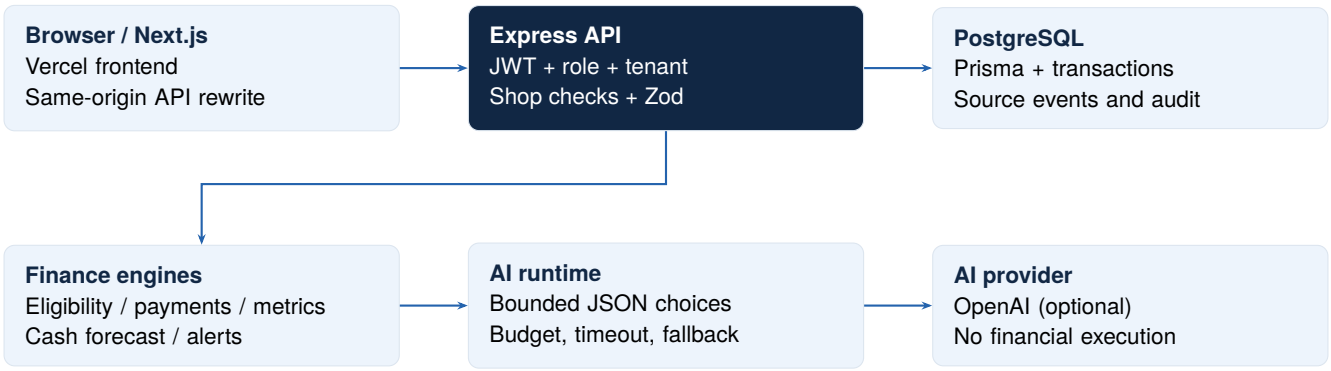
The finance copilot operates in a sandbox: financing offers and money movement are simulated. No external lender is contacted and no bank transfer is executed. The business-management platform supplies the records; this document focuses on the Topic E finance workflows and their limits.

Application: `https://bizwizard-lake.vercel.app/`
Repository: `https://github.com/mahfuz-saim/bizwizard`
Reviewer access is required for the private repository.

02 / ARCHITECTURE

System Architecture

Application components, data flow and trust boundaries.



Trust boundaries: browser | application/VPS | database | optional external providers

Security design and data flow

The browser sends authenticated requests to the frontend's same-origin /api/proxy/v1 rewrite. Next.js forwards to the configured backend. Express verifies the token and required role, checks tenant membership and employee shop assignment, validates the request, then invokes a scoped service. CORS is not the authorization boundary.

Layer	Responsibility
Presentation	Next.js / React / Tailwind; dashboard workflows, light/dark themes, loading states and owner/employee controls. Hidden buttons are convenience, not access control.
Finance service	Matching, cash flow, analysis, agent ledger and simulator modules. Pure TypeScript engines calculate money and constraints; service transactions persist approved state.
Persistence	One production PostgreSQL database managed through Prisma migrations. Sales, debts, lots and expenses feed finance records; policy versions, decisions, runs and evaluation ledgers support traceability.

The disposable local simulator_test database is test infrastructure, not a second required production database. In-app demo workspaces are separate, scoped shops copied from a source shop; a virtual clock isolates simulated dates. Source history must remain unchanged through copy, run and reset.

Key record families: FinancingOffer / Application / LoanSchedule; CashAccount / Transaction / Commitment / Alert / TreasuryAction; Insight / Scenario / CapitalDecision; AiAgentPolicy / AiDecision / AiRun / FinanceEvaluation. This is not full double-entry or bank-reconciled accounting. Upload and push integrations are separate backend services, outside the finance path shown above.

03 / FINANCING

Financing Algorithms

Eligibility, affordability, offer ranking and bounded negotiation.

Stage	Rules enforced by code
1. Profile and completeness	Aggregate the shop history: revenue, COGS, operating expense, receivables, inventory, debt burden and cash. Matching requires at least 60% coverage of its 90-day sales window.
2. Eligibility gate	Check amount, tenor, business age, revenue, debt/revenue, sector and required-document flags. NEAR_MISS means exactly one soft threshold missed by at most 20%; it is not automatic approval.
3. Rank feasible products	Weighted score: affordability 30%, cost 25%, liquidity 20%, collateral 15%, flexibility 10%. AI enrichment is bounded to +/-15 score points; the blend is 60% deterministic and 40% bounded AI.
4. Price and negotiate	Initial offers pass product and affordability guards. Pricing and the accept/counter/reject policies are deterministic, with at most three rounds; they are not an LLM-to-LLM negotiation.
5. Owner confirmation	Acceptance checks shop access, offer state and expiry, then saves the application and rounded schedule in one database transaction. Repeated acceptance reuses it; affordability is not recalculated at this stage.

Money and affordability

$$
\begin{aligned}
\mathrm{FCF} &= R - C - O - D, \\
\mathrm{EMI}_{\mathrm{screen}} &= 0.50\,\max(0, \mathrm{FCF}), \\
\frac{\mathrm{PMT}}{\mathrm{FCF}} &\le 0.35 \quad (\mathrm{FCF} > 0), \\
\mathrm{PMT} &= \begin{cases} P/n, & r = 0, \\ Pr/\left(1 - (1+r)^{-n}\right), & r > 0, \end{cases} \qquad r = \frac{i_{\mathrm{annual}}}{1200}.
\end{aligned}
$$

FCF is free cash flow; COGS is cost of goods sold; EMI and PMT refer to monthly installments. R is monthly revenue, C is COGS, O is operating expense, D is existing debt service, P is principal, n is the number of installments, and the annual rate is a percentage.

The 50% screening capacity and 35% borrower policy ceiling are separate checks. AI enriches product ranking; deterministic policies price and negotiate offers. Currency boundaries round to two decimals, with the last installment absorbing residual principal. Submission is not disbursement.

Model limitations. Missing lot costs are estimated at 70% of line sales value. Credit risk uses rules of thumb, not a validated bureau score. The effectiveApr helper annualizes total cost divided by principal; it is not a regulatory annual percentage rate (APR) based on the timing of payments. Quotes are simulated, not real credit commitments.

04 / CASH FLOW AND ANALYSIS

Cash Flow and Financial Analysis

Cash synchronization, projection, alerts and financial analysis methods.

Cash synchronization and projection

PostgreSQL triggers keep cash records in step with non-simulated paid sales, debt payments and expenses, including edits and deletions. Demo shops use a separate simulator-managed ledger. Backfill adds missing historical cash entries without duplicating them or replacing manual entries. Settling a supplier obligation links an existing matching purchase; it does not issue another payment.

The projection builds daily historical inflow/outflow series, estimates recent levels and day-of-week seasonality, applies a clipped trend, and adds scheduled commitments and probability-weighted receivables. It rolls balances forward for 90 days and identifies the trough, first shortfall and buffer days.

Nine alert conditions	Behavior
Shortfall within 14 days; buffer below target	Prioritize potential liquidity pressure; low-history rules suppress inappropriate confidence.
Commitment due soon; debt older than 30 / 60 days	Show dated obligations and aging receivables. The 30- and 60-day checks are separate conditions of the overdue alert family.
Idle cash; revenue decline; expense spike; insufficient data	Flag idle balances, a 7-day revenue decline relative to the prior average, expenses above twice their category median, or fewer than 14 ledger days.

Analysis and hypothesis verification

The analysis module computes margins, product contribution, working capital, breakeven and concentration from the selected shop’s records. It checks an explanation by subtracting its contribution from the observed gap. A claim is VERIFIED only when at most 10% of the nonzero gap remains unexplained. This checks whether the numbers fit, not whether one event caused another.

Scenario levers change price, collections, operating cost or product mix through an explicit simplified model. Approved capital decisions freeze an observation window; 30-day before/after profit observations exclude future rows and remain non-causal. Treasury collection evidence links unique existing payments and is gross cash, not incremental profit.

Assumptions: collection probabilities and scenario elasticity are heuristics. Receivable age can stand in for overdue age. Scheduled commitments can overlap ordinary historical expenses. Cash accounts are not externally reconciled; historical supplier obligations must be recorded explicitly.

Worked example: from a sale to evidence

Illustrative flow, not a measured result. A paid sale of BDT 1,000 is saved in a non-demo shop. Its cash entry is synchronized automatically and feeds the next projection. If a cash-buffer alert appears, the owner can review a proposed collection plan before approving it. Approval alone adds no cash: recording a collection must link an existing valid payment. The saved plan and payment link let the reviewer inspect what was approved and what was actually recorded.

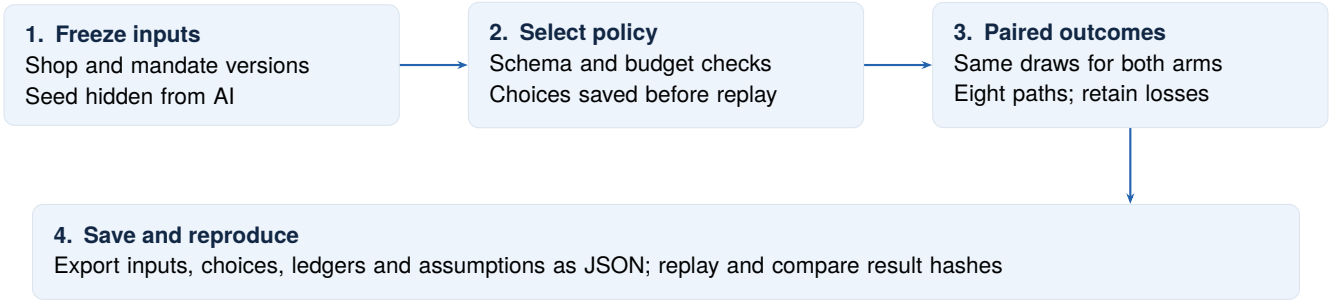
05 / AI RUNTIME AND EVALUATION

AI Runtime and Evaluation

Structured model choices, deterministic validation and paired sandbox measurement.

Each AI call requests JSON, checked with Zod against the expected fields and allowed values. One repair retry is permitted within the time limit. Missing keys, invalid output, exhausted budgets and timeouts use a clearly labelled rules-based fallback. Calls reserve part of the business's monthly AI budget. The system attempts to log the model, prompt version, tokens, estimated cost and status; a logging failure does not block the feature.

Shop-specific paired evaluation



The snapshot requires 60% sales-history completeness and at least 14 cash-history days. A repeatable-read database transaction captures a consistent view of quotes, product economics, commitments and versioned owner policies (mandates). The AI cannot choose the random seed or future outcomes. A unique snapshot key and a single database claim prevent duplicate paid calls; unchanged daily inputs reuse the saved evaluation.

Role	Paired sandbox metric
Lender	Net cash return over the loan term after funding costs, repayments and modeled recovery. AI and fixed-rate rules use the same loans, exposure limits and random default outcomes.
Borrower	Full-term borrowing cost on equal-principal feasible quotes; baseline selects lowest quoted APR. Both sides assume repayment in full, independently of the lender experiment.
Treasurer / analyst	30-day closing cash versus delayed-collection rules; modeled capital gross margin versus equal allocation among products ranked by revenue. Each role is a separate experiment.

Evaluation limitations. Eight paired paths do not establish a real-world treatment effect. Quotes use captured current terms rather than reconstructed pre-negotiation contracts. Missing data stays pending, and losses and ties are retained. Sandbox results are reported separately from real-world outcomes.

06 / VALIDATION

Validation Results

Live-provider benchmarks and isolated application execution tests.

Recorded on 18 October 2026 (UTC), from clean revision `8f394d2d074b`. These are saved results, not a fresh test of the current release. Each JSON report includes the full revision and source hashes.

Live AI versus pinned rules: three synthetic risk cases

The differences below correspond to default-risk inputs of 5%, 20% and 50%, in that order.

Role	AI / rules / tie	Observed difference in favour of AI
Lender	1 / 2 / 0	BDT -4,000.00, -1,000.00, +5,000.00
Borrower	3 / 0 / 0	BDT +4,099.31, +4,099.31, +4,099.31
Treasurer	0 / 0 / 3	BDT +0.00, +0.00, +0.00
Analyst	0 / 3 / 0	BDT -60.00, -60.00, -60.00

Across 12 role/case comparisons: **4 AI wins, 5 rules wins and 3 ties**. Borrower cost fell by BDT 4,099.31 in each case; analyst modeled returns were BDT 60 lower. All three provider calls succeeded without fallback using gpt-4o-mini; total estimated API cost was USD 0.000273. Counts are not a combined financial return.

Separate shop-specific live sandbox run

Role	Baseline (BDT)	Participant (BDT)	Difference
Lender	665.94	265.22	-400.72
Borrower	705.82	423.28	+282.54
Treasurer	1,484,037.23	1,484,037.23	+0.00
Analyst	46,761.23	46,761.23	+0.00

These are means across eight paired paths: lender and borrower use their loan terms; treasurer and analyst use 30 days. The live policy succeeded without fallback; its estimated API cost was USD 0.000155. Offline replay produced the same result hash, a fingerprint of the consistently ordered outcome data. Positive differences favour the participant; borrower differences measure cost saved, while other roles measure extra return or cash.

Database execution evidence

11 tests passed, zero failed or skipped across five suites, plus successful migration application. Coverage includes transactional cash events, supplier evidence, inventory/alert behavior, finance authorization and budgets, matching/application concurrency, isolated demo copy/run/reset, and frozen paired evaluations.

> The standalone benchmark is not a replay of the full multi-agent negotiation pipeline. Fixture mode is not AI. Integration evidence is backend service/loopback HTTP evidence, not a browser video, production penetration test or real-world financial result.

07 / DEPLOYMENT AND REPRODUCTION

Deployment and Reproduction

Prerequisites, execution commands and the reviewer workflow.

Evaluation commands and prerequisites

```
# From server/ after npm ci
npm run benchmark:finance
npm run benchmark:finance -- --live
npm run evidence:finance
npm run evidence:finance -- --replay EXPORTED_EVALUATION.json
```

All commands need Node.js 24, npm, Git and installed dependencies. Offline benchmarks and JSON replay need no database or AI key. Only –live uses your key from server/.env.benchmark or the environment. Database evidence needs local PostgreSQL at 127.0.0.1, database simulator_test and EVIDENCE_DATABASE_URL; it creates a fresh schema, applies migrations and disables external AI. See the README for setup steps.

Application deployment

Frontend: Vercel, root client, Next.js preset; set NEXT_PUBLIC_API_BASE_URL to the backend /api/v1 endpoint. **Backend:** main-branch CI builds an artifact and deploys over restricted SSH. The installed script installs dependencies, generates Prisma, starts backup, applies migrations, switches releases and checks readiness. VPS secrets and services must already be configured; frontend deployment is separate.

Use migrate deploy, not schema-only db push: migrations install cash triggers, supplier-obligation fields and evaluation storage. Owners can backfill existing cash history. Code rollback does not undo database migrations.

Review workflow

Use a synthetic account or demo workspace. Inspect a Cash Flow projection and alert; run financing matching and review negotiation and an approved application; inspect verified analysis. In AI Agent Ledger, run a matched evaluation, download it and replay the JSON. See the Security Self-Assessment for permissions and risks.

References and supporting material

Requirements: FinTechathon International Track appendix, Topic E and submission checklist (pages 2-3). **Setup:** root README. The linked reports below require repository access.

Report in evidence/submission/	What to verify
benchmark-results.json	Offline fixture comparison; no AI call.
benchmark-live-results.json	Three live-provider cases, choices, costs and results.
sandbox-operation-logs.json	Migrations, five integration suites and numeric checks.
matched-evaluation.json	Shop snapshot and rules-based paired evaluation.
matched-evaluation-live.json	Live AI choices and eight paired outcome paths.
matched-replay-verification.json	Offline reproduction of the live evaluation.